1 Overview

The design of the runtime will be based on the 1992 reference [1] plus the augmentation to use Apply/Eval approach from the 2006 reference [2]. We will also use a single unified stack.

2 Frame Types

Using one unified stack by marking boundaries of parts of the stack by frames means no standalone items (such as arguments) will exist on its own in the stack; instead they will be described by frame headers. When accessing the stack, we will always be able to inspect a frame header that is present on top of the stack before manipulating the stack. The variants of frame are:

- Continuation Frame
- Application Frame
- Update Frame

For each of the above types of frames, a stack frame header is embedded (pointer to some meta-info describing items associated with the frame that are also pushed earlier onto the stack). The stack frame meta-info is used by the garbage collector (GC) to discriminate pointer/non-pointer types (GC will use this to trace live objects instead of using a separate stack as roots of live objects as in the approach of the 1992 reference [1]).

These frames are produced and consumed during certain state transitions that are mentioned in the following sections of this document.

3 Continuation Frame

3.1 triggered by

start of evaluation of a case expression

3.2 actions to the stack

push each local env variable that are live across the continuation

push a Continuation Frame that is embedded with:

- stack frame info containing previously inserted items onto the stack
- return address/vector corresponding to the alternatives

3.3 consumed by

an eventual return of constructor transfers control to the continuation (for example, one can possibly use the convention of return vector jump indexed by the constructor variant), where it pops from the stack:

- continuation frame
- local vars associated with the continuation frame (these are read and used for the continuation)

before proceeding to execute the remainder of the continuation

4 Application Frame

4.1 triggered by

a function application, and before evaluation of the function to weak head normal form

4.2 actions to the stack

push each argument into the stack

push an Application Frame that also embedded with:

- stack frame info for previously inserted items onto the stack

4.3 consumed by

before an enter of a closure/function, an arity check is run using the application frame info that already encodes available arguments on the stack

different scenarios to consider:

1) if the closure finds over-saturation of arguments:

copy and remove the function's required arguments off the stack and adjust the application frame to encode the remaining arguments on the stack, before jumping to the function's entry code with the arguments (eg: a possible calling convention could be leaving those arguments on the stack)

2) if the closure finds exact number of arguments avaialable from the application frame:

pop the application frame off the stack, copy and pop all the arguments (associated with the application frame) off the stack, then jump to the function's entry code with the arguments

3) if the closure finds under-saturation of arguments:

create a new partial application (PAP) closure on the heap taking the existing function closure and available arguments from the stack as payload on the heap, pop the application frame off the stack and all the arguments off the stack, and return the new PAP closure (for example, by saving its address in a designated register)

5 Update Frame

5.1 triggered by

when thunk / updatable closure is entered for the first time

5.2 actions to the stack

push an Update Frame that also embedded with:

- stack frame info
- address of the thunk / updatable closure in the heap

5.3 consumed by

either:

- a return of constructor, or
- a completion of a partial application (where a previous removal of an application frame from the stack happened)

sees the update frame on top of the stack

we read the address of the thunk associated with the Update Frame and update the thunk with a target closure by overwriting the thunk with an indirection closure

notes about indirection closure:

it is a closure with:

- 1 payload field that is the address of the target closure
- entry code that jumps to the target closure (its entry code)

this means a thunk / any closure needs to be atleast 2 words in size (1 payload field + a pointer to its entry code)

the indirection closure can be removed during a GC sweep since we can use the target closure pointer and remove the intermediate indirection

6 States

a tuple of:

- code
- $s \equiv$ unified stack: containing a frame of type: continuation/update/application plus any relevant payloads associated with the frame
- $h \equiv$ heap: for locals, closures
- $\sigma \equiv$ globals: for top-level variables/binders

7 State Transition: eval case expression

we enqueue live local variables into the stack along with a continuation frame

```
eval (case e of alts) ρ, s, h, σ

⟹

eval e ρ, s', h, σ

where:

s' = (cf:vs) ++ s

vs = local live vars for alternatives (
    equivalent to providing ρ for continuation)

cf = continuation frame (layout-aware of its 'vs
    ' payload)
```

8 State Transition: eval let/letrec expression

this is largely same as original transition in reference [1]

for letrec:

```
eval (letrec x1 = vs1 \π1 xs1 ↦ e1; ...; in e)
    ρ,
s,
h,
σ

⟹

eval e ρ', s, h', σ

where:

  ρ' = ρ[x1 ↦ addr a1, ...]

  h' = h[a1 ↦ (vs1 \π1 xs1 ↦ e1)
          (ρrhs vs1);
          ...]

  ρrhs = ρ'
```

9 State Transition: eval constructor

```
eval (c xs) ρ, s, h, σ

⟹

ReturnCon c (val ρ σ xs), s, h, σ
```

9.1 intermediate state: ReturnCon

```
ReturnCon c ws, ((cf:locals)++s), h, σ

where:

  cf = continuation frame

⟹

eval e ρ', s, h, σ

where:
  e = expression to eval after jump to
      continuation (eg: jump to fr.return_vector
      [offset])

  offset = computable based on constructor
      variant 'c' (or tag depending on
      convention)

  ρ' = ρ ∪ {vs ↦ ws}

  ρ = popped 'locals' from continuation frame
```

for default pattern match: don't need to add additional {vs ↦ ws} binding to local env.

for match-call variable binder pattern: create a closure in the heap for the constructor 'c', introduce a binding to the created closure in the local env., use 'ws' as payload to the closure in order to bind to the constructor parameters

10 State Transition: eval literal

similar logic as constructor

```
eval k ρ, s, h, σ

⟹

ReturnInt k, s, h, σ
```

eval of a variable bound to a primitive value:

```
eval (f {}) ρ[f ↦ Int k], s, h, σ

⟹

ReturnInt k, s, h, σ
```

10.1 intermediate state: ReturnInt

```
ReturnInt k, (cf:locals)++s, h, σ

where:

  cf = continuation frame

⟹

eval e ρ, s, h, σ

where:

  e = expression to eval after jump to
      continuation

  ρ = popped 'locals' from continuation frame
```

handling alternative with match-all variable binder x:

create additional local variable that binds to Int primitive:

```
ρ = popped 'locals' from continuation frame
        ∪ {x ↦ Int k}
```

11 State Transition: eval builtin operator

for example, binary operator ⊕ with primitive int parameters:

```
eval ⊕{x1, x2}
  ρ[x1 ↦ Int i1, x2 ↦ Int i2],
s,
h,
σ

⟹

ReturnInt (⊕ i1 i2), s, h, σ
```

12 State Transition: eval function application

```
eval f xs ρ, s, h, σ

⟹

// construct the application frame before eval
    of f to weak head normal form
eval f ρ, (af:xs)++s, h, σ

where:

  af ≡ application frame

  xs ≡ arguments associated with function
       application
```

after eval of the functional value f:

```
eval f ρ, (af:xs)++s, h, σ

where:

  f ≡ a value that has already been evaluated to
      weak head normal form and is a functional
      type with addr 'a' in the heap

  af ≡ application frame

  xs ≡ arguments associated with the application
       frame

⟹

// exact arguments saturation
if arity(f) = len(xs):

enter a with xs, s, h, σ

---

// over-saturation of arguments
if arity(f) < len(xs):

enter a with xs1, (af':xs2)++s, h, σ

  where:

    xs1++xs2 = xs, len(xs1) = arity(f)

    af' ≡ adjusted application frame describing
         over-saturated/remaining arguments 'xs2'

---
// under-saturation of arguments
if arity(f) > len(xs):

// pop application frame off the stack
//
// give control to whatever is next on the stack
     (eg: continuation / update)
return PAP closure, s, h', σ

  where:

    // create a PAP closure with address a' in
        the heap and take available args and
        function from the original stack into
        closure's payload
    h' ≡ h[a' ↦ (f':vs1 \n vs2 ↦ f' (vs1++vs2)
        ) a:xs]

    // 'xs' are originally arguments on the
        stack but transferred in as free
        variables in the closure's payload
    len(xs) = len(vs1)

    // 'vs2' are the remaining arguments for
        function f
    len(vs1) + len(vs2) = arity(f)
```

13 State Transition: eval updatable closure

todo

References

[1] Simon L. Peyton Jones. Implementing lazy functional languages on stock hardware: The Spineless Tagless G-machine. *Journal of Functional Programming*, 2(2):127–202, 1992.

[2] Simon Marlow and Simon Peyton Jones. Making a fast curry: push/enter vs. eval/apply for higher-order languages. *Journal of Functional Programming*, 16(4-5):415–449, 2006.